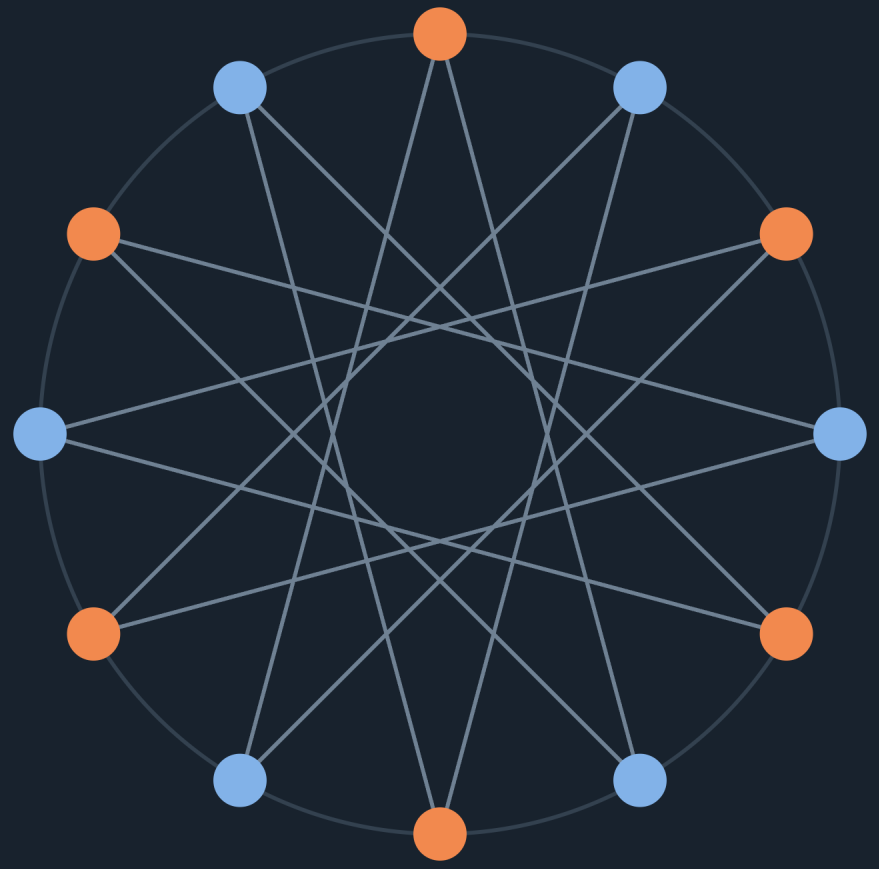
COMPUTATIONAL INTELLIGENCE · LAB 3

Seating children in a circle

A max-TSP-like problem, solved with a hill climber and a genetic algorithm — from the lab's first versions to the proven optimum.

Tommaso Garzaro



THE PROBLEM

Seat them so neighbours differ the most

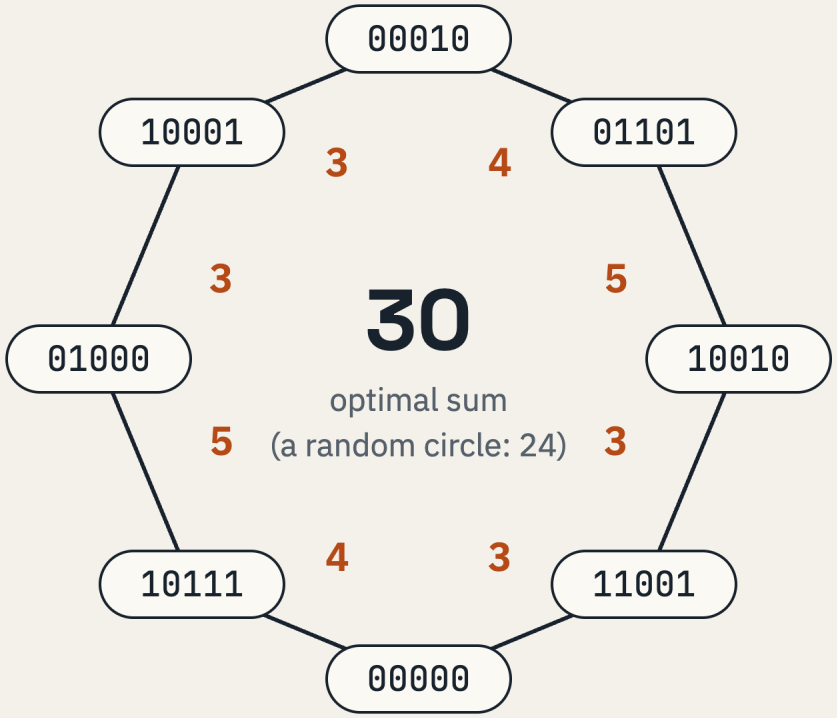
N children, each with **C yes/no characteristics** — a profile such as 01101.

Distance between two children = **Hamming distance**, the number of characteristics on which they disagree: 00010 vs 01101 → 4.

Goal: arrange them in a circle that maximises the sum of distances between neighbours, last–first included.

```
max over π of
Σ d( π[k], π[(k+1) mod N] )
```

A travelling-salesman tour, but maximised: “max-TSP” on a Hamming metric. Example on the right: 8 children, C = 5.



THE PROBLEM

Too many circles to try them all

Rotations and reflections are the same circle, so there are **(N − 1)! / 2** distinct seatings.

181,440

circles at N = 10

About the largest size where exhaustive enumeration (exact_best) still works.

283

digits in the count at N = 160

The benchmark instance used throughout: N = 160, C = 5, seed 2027.

748

its proven optimum

Known only much later. For most of the project, nobody could say how far from the best a result was.

So we need heuristics: a **hill climber** and a **genetic algorithm**.

DISCLAIMER 1 · OPTIMISATION

My approach to the lab

The lab asked for a solution (HC and GA) and I kept measuring and improving. Every version is an **era**: registered with its phase and parent.

A

The lab's solution

Phases 01–02

`hc-legacy`
`ga`

→

B

Measure, then improve

Phases 03–06: fair benchmark

`hc`
`hc-i`

→

C

Know the optimum

Phase 07: exact solver

`exact`

→

D

Insight back into HC and GA

Phases 08–14: layers S1–S6

`hc-p-grasp`
`ga-e-seed`

Equal time. Same instances, same seeds, 0.2 s per run for every algorithm.

One switch per idea. A layer switched off reproduces its parent exactly (tested).

Runnable. python -m circle_optimization run hc-legacy

DISCLAIMER 2 · USE OF AI

AI was used deliberately and widely

Validating and refining ideas

Challenging and sharpening a hypothesis before building it, understanding its theory, choosing the experiment that would confirm or reject it.

Building and testing fast

Modules, tests and benchmark runs implemented quickly, so every idea became measured evidence before the next step.

Writing documentation

Engineering records, READMEs and the notebook narrative.

Not “vibe-coded”

The ideas and the engineering, the approach, the project structure, the modules, the solutions adopted or rejected came from me, directly or after deep dives (with AI) on GA and CI topics. Each step is recorded with its motivation and measured evidence in HISTORY.md.
This is not a one-shot prompt at all: the whole process took roughly 20 hours of work.

Models · GPT-Sol 6.1 (x-high), Claude Opus 5.5 (x-high)

Harness · Opencode, Claude Code

01

The lab’s first approach

How a circle is stored and scored, the two moves, the hill climber and the genetic algorithm: eras hc-legacy and ga.

PHASE 00–01 FIRST APPROACH · REPRESENTATION

A circle is a tuple of child indices

`(2, 3, 0, 1)` rotated

`(0, 3, 2, 1)` reflected

`(0, 1, 2, 3)` as is

→

`(0, 1, 2, 3)`

Child 0 first, the smaller direction kept: one tuple per circle, usable as a cache key.

	0	1	2	3	4	5	6	7
0	0	1	4	4	1	3	3	2
1	1	0	3	3	2	2	4	1
2	4	3	0	2	5	3	3	2
3	4	3	2	0	3	1	3	2
4	1	2	5	3	0	2	2	3
5	3	2	3	1	2	0	2	3
6	3	4	3	3	2	2	0	5
7	2	1	2	2	3	3	5	0

Distances read from an N × N matrix

Built once in O(N²C). Scoring a circle is then N table lookups. This is the 8-child example of the problem slide.

THE FUNCTIONS

`distance_matrix(profiles)`

N × N Hamming table, computed once.

`canonical_circle(genome)`

Picks one direction: (0,3,2,1) → (0,1,2,3).

`circle_fitness(genome, D)`

Sum of D[g[k]][g[k+1]], last–first included: O(N).

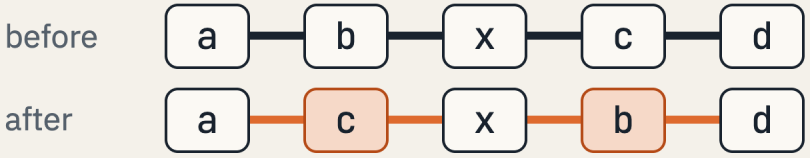
`exact_best(D)`

Enumerates every circle: the true optimum, only up to N ≈ 10.

PHASE 01–02 FIRST APPROACH · MOVES

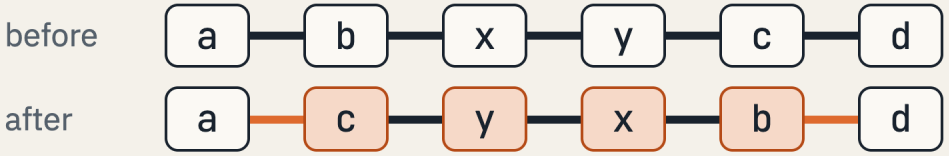
Two moves, each scored in O(1)

Swap(i, j) — exchange two children: 4 edges change




```
Δ = new - old, over the ≤ 4 touched edges
```

Inversion(i, j) — reverse a segment (2-opt): 2 edges change




```
Δ = D[a][c] + D[b][d] - D[a][b] - D[c][d]
```

THE FUNCTIONS

`swap_delta(circle, D, i, j)`

Score change of a swap from the touched edges only.

`inversion_delta(circle, D, i, j)`

Four lookups, whatever N is.

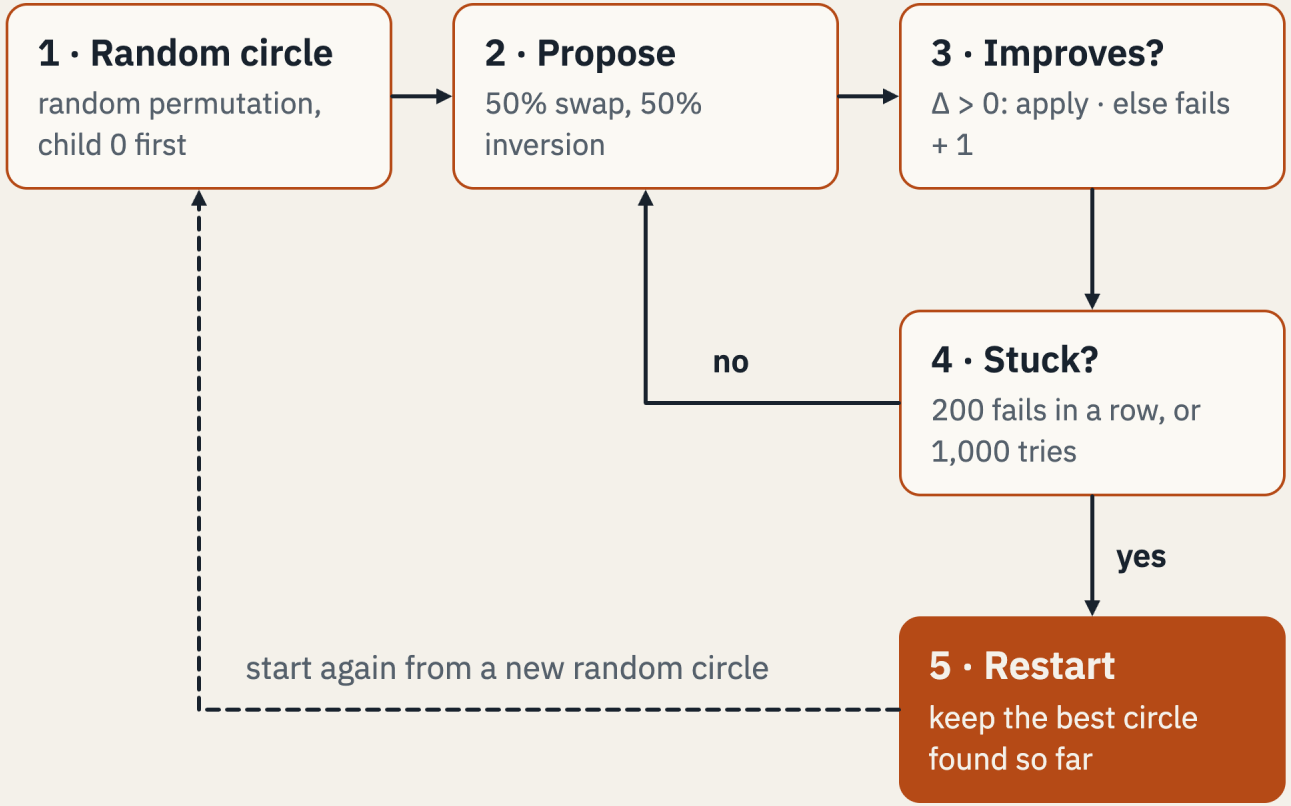
`mutate(genome, rng)`

GA version: a swap or an inversion with probability ½ each, returned as a new canonical tuple.

> A rejected proposal costs 4 lookups, not a full O(N) rescore, so the HC can try far more moves.

`hc-legacy` FIRST APPROACH · HILL CLIMBER

Propose, keep only if better, restart



THE FUNCTIONS

`_run_hill_climber(D, rng, …)`

The loop on the left: strict improvement, restarts, budget, deadline.

`random_individual(n, rng)`

Step 1: a random canonical circle.

`uniform_swap_positions(n, rng)`

Step 2: two random positions ≠ 0.

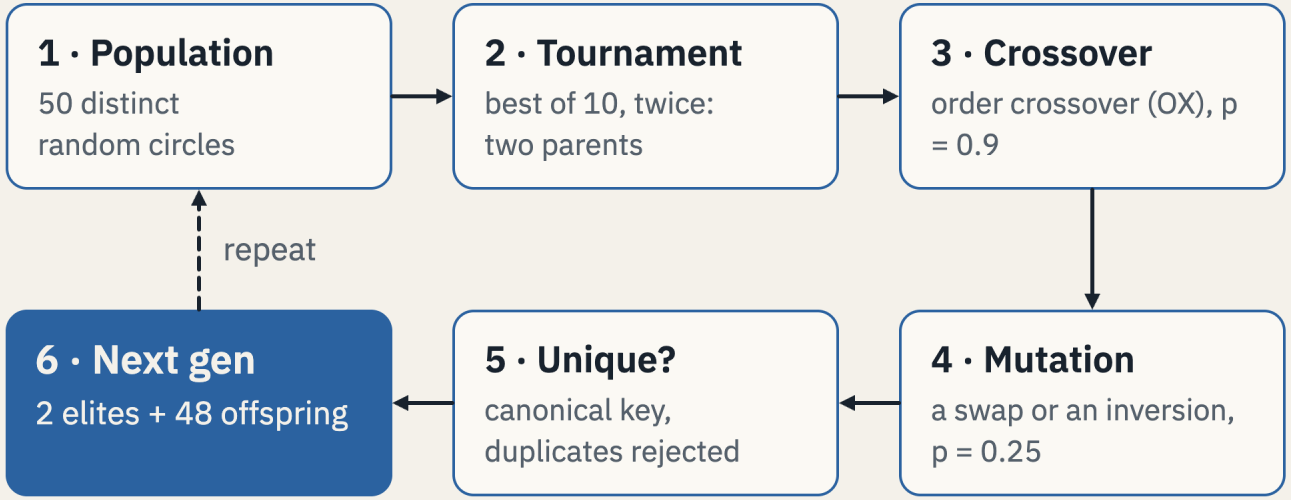
`swap_delta · inversion_delta`

Step 3: Δ without building the circle.

`inversion_rate=0.5 · stall_limit=200`
`attempts_per_restart=1000`

ga FIRST APPROACH · GENETIC ALGORITHM

One generation: select, cross, mutate



Fitness cache

Each canonical circle is scored once and stored: the budget counts distinct circles.

THE FUNCTIONS

`evolve(population, rng, …)`

The generation loop; scoring and operators are injected, so every later GA reuses it.

`tournament_selection(…, k)`

Step 2: the fittest of k sampled circles.

`order_crossover(a, b, rng)`

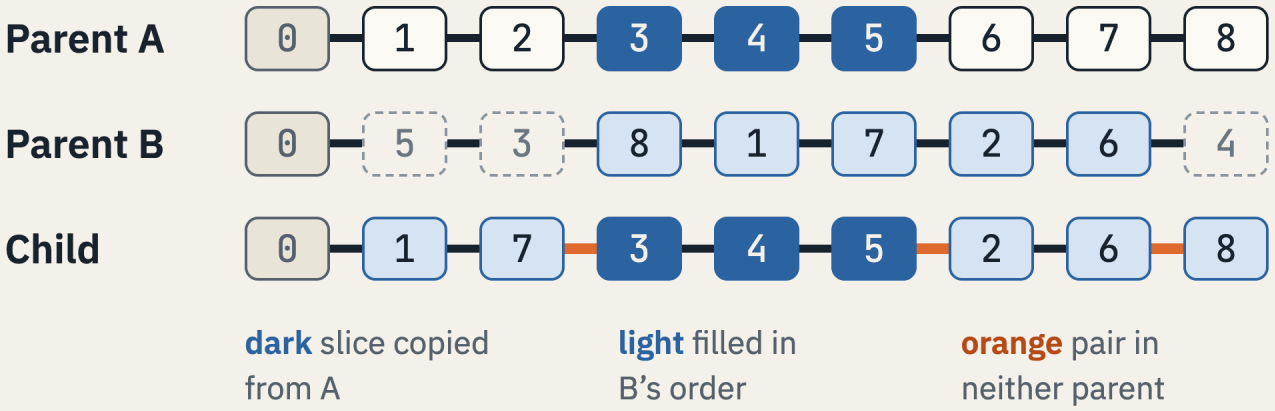
Step 3, next slide.

`mutate · FitnessEvaluator`

Step 4, and the cache keyed by canonical tuple.

`order_crossover` FIRST APPROACH · CROSSOVER

OX inherits order, not neighbours



3 of 9 neighbour pairs are "invented": 7–3, 5–2, 6–8

The score depends on who sits next to whom, but OX passes on relative order. This is fixed in era 12.

1 Copy a random slice of parent A, here 3 4 5.

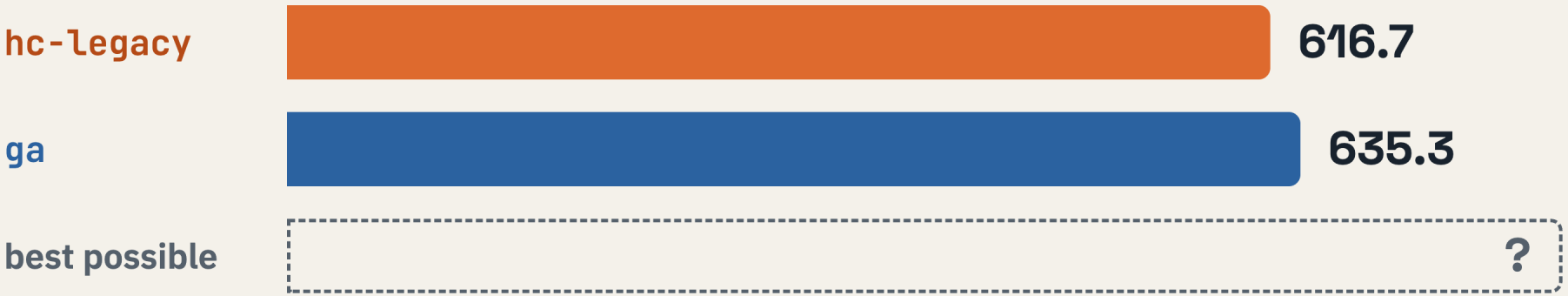
2 Read parent B cyclically from just after the slice, skipping genes already used: 2, 6, 8, 1, 7.

3 Fill the free positions in that order: after the slice first, then from the start.

Every child appears exactly once, so the result is always a valid circle; child 0 stays fixed as the anchor.

N = 160 · C = 5 FIRST APPROACH · RESULTS

First results: decent, but how good?



Mean score over 3 seeds, 0.2 s per run, on the benchmark instance. Bar length = score.

Why is the GA ahead at this size?

Where is the HC failing?

How far are we from the optimum?

02

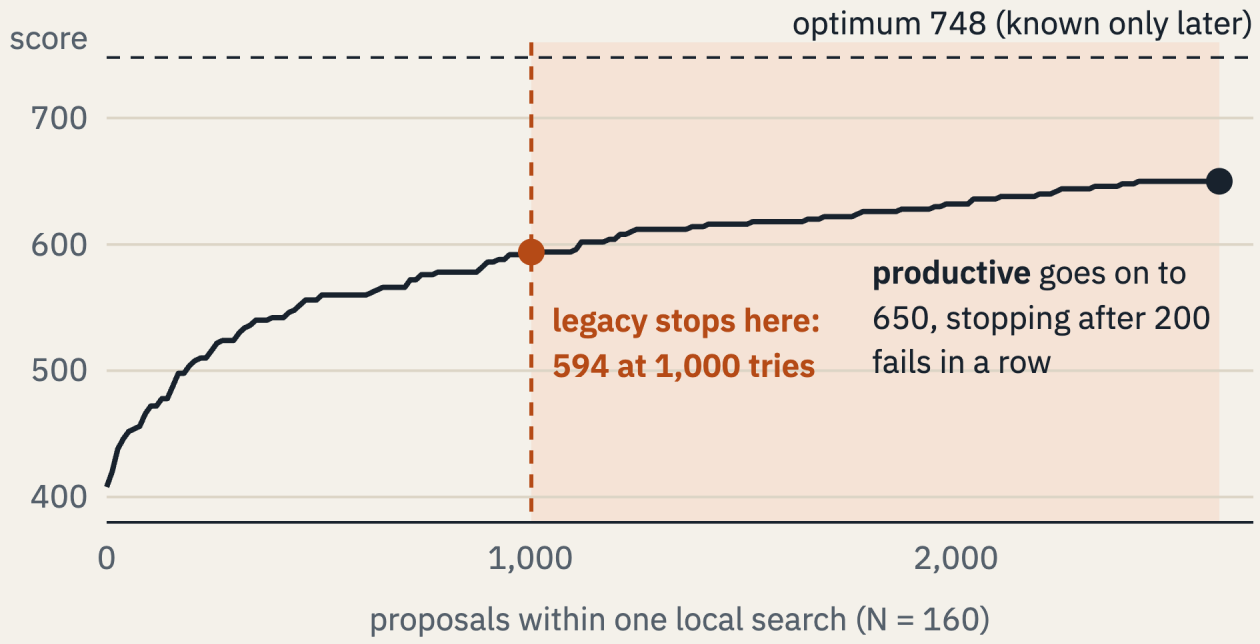
Eras, one idea at a time

For each step: the problem observed, what the enhancement does in practice, the Python functions behind it, and its measured effect.

04 restarts | 07 exact solver | S2 profile state | S1 splice | S3 reduced costs | S4 edge genome | memetic | S5 propagation | S6 greedy seeds

hc · hc-i ERA 04 · PRODUCTIVE RESTARTS, INVERSION FOCUS

The HC restarted while still improving



19 / 20

legacy local searches ended at the 1,000-proposal cap, not because they were stuck.

`restart_policy="productive"`

Restart only after 200 failures in a row; every improvement resets the counter.

hc: 616.7 → 666.7

`run_inversion_hill_climber(…, 0.8)`

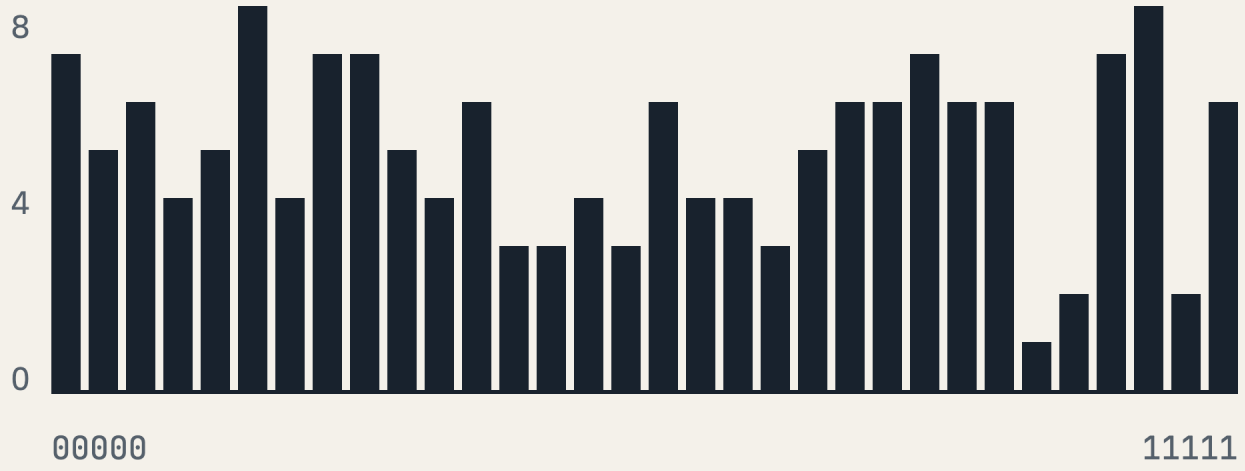
80% inversions, 20% swaps: 2-opt suits tours.

hc-i: 668.0, the best classic HC

exact ERA 07 · THE KEY OBSERVATION

160 children, only 32 different profiles

With C characteristics there are at most 2^C profiles: 32 for C = 5. Most children share one.



Children per profile on the benchmark instance: one bar per distinct profile, from 1 to 8 children each.

Same profile = interchangeable

So a circle is an **m × m table**: how many neighbouring pairs join profiles p and q. For circle A B A C C B:

	A	B	C
A	·	3	1
B	3	·	1
C	1	1	1

Valid circle ⇔ each profile p has degree 2·n_p and the table is connected.

`aggregate_profiles(profiles)`

Profiles, counts, m × m distances.

exact ERA 07 · PROFILE-AGGREGATED MILP

An exact solver whose size ignores N

1 · Aggregate
`aggregate_profiles`
N children → m ≤ 2^C profiles and their counts.

→

2 · Warm start
`multi_start`
Greedy matching by distance, then 2-exchange local search.

→

3 · LP + MILP
`solve_profile_milp`
HiGHS; cuts added lazily; reduced-cost fixing.

→

4 · Back to a circle
`euler_sequence`
Euler circuit on the table, then children assigned.

```
max  Σ d[p][q] · x[p][q]
s.t. Σ x[p][q] + 2·y[p] = 2·n[p]  degree
     Σ x across any cut S ≥ 2  connected
     x, y integer  y[p] = loops in p
```

m(m−1)/2 + m variables, whatever N is.

1,000,000 children (C = 8) proven optimal in 0.7 s; N = 160 in 78 ms.

Not my invention: I studied how exact solvers approach the problem, to borrow ideas for HC and GA.

OPTIMUM = 748 ERA 07 · THE REVEAL

The classics were 10–18% below optimum



Mean gap to the proven optimum, benchmark instance, 0.2 s × 3 seeds.

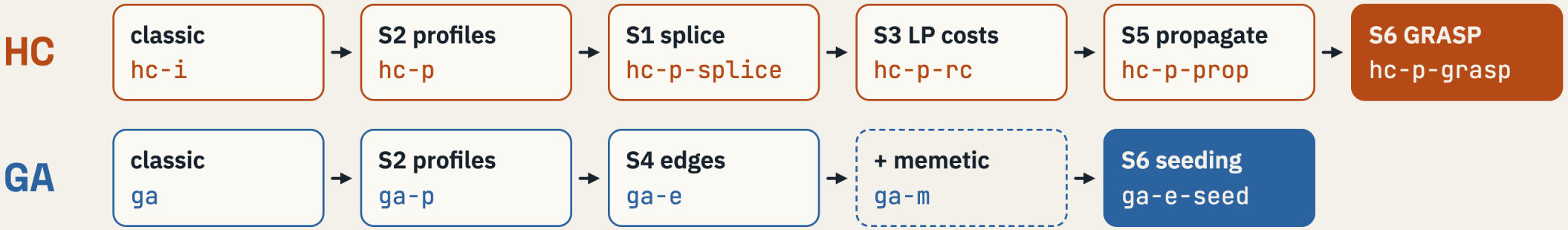
61–98%
of the gap closed by moves on the profile table alone, starting from their final circles

11–62%
of their edges join profile pairs that the LP proves sub-optimal

Missing moves and information, not time

PHASE 08 STAGE D · ERAS AND LAYERS

Each idea became a switchable layer



The same loop for every layer

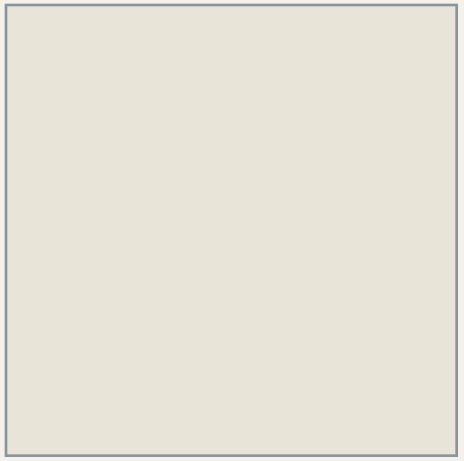
1 Write the idea and its expected effect → **2** Implement it as one module + one switch → **3** Test: switched off, it equals the parent → **4** Measure against the parent at equal time → **5** Keep it, or record why not

hc-p · ga-p LAYER S2 · PROFILE STATE (PHASE 09)

S2: store profiles, not children

children	0	35	146	17	66	31	127	116
profiles	2	6	23	25	28	28	24	21

Real start of a circle: children 66 and 31 both have profile 28 (00111).



N × N = 25,600 cells



m × m = 1,024

Areas to scale. At N = 100,000 the matrix needs 10^10 cells; the profile table stays at most 2^C × 2^C.

PROFILESTATE

`.from_profiles(profiles)`

Aggregate once; m × m table as tuples.

`.sequence(genome)`

Child circle → profile ids.

`.score(sequence)`

circle_fitness on the m × m table.

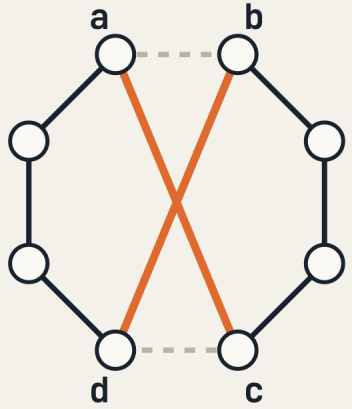
`.circle(sequence)`

Back to children: k-th visit of p ← p's k-th child.

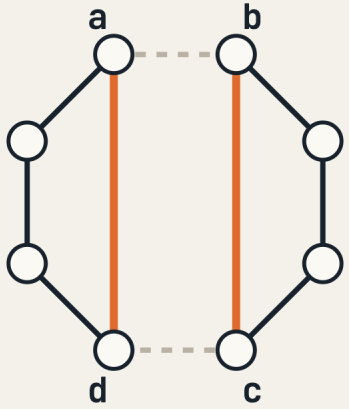
> Same seed and budget: hc-p follows hc move for move (tested).

`hc-p-splice` LAYER S1 · SPLIT-AND-SPLICE (PHASE 10)

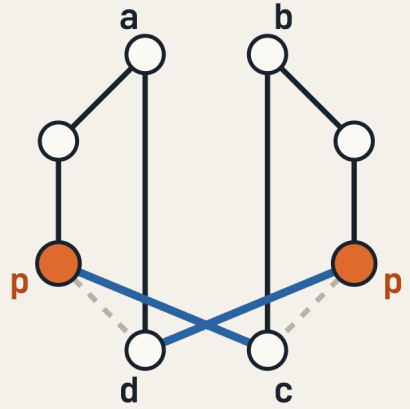
S1: the other way to reconnect two cuts



1 · Inversion: cut a–b, c–d, add a–c, b–d → still one circle



2 · Splice: add a–d, b–c instead → two separate loops



3 · Re-join at a profile p in both loops: same pairs, Δ = 0

`splice_delta(…)`
O(1): D[a][d] + D[b][c] − D[a][b] − D[c][d]

`shared_profile(…)`
A profile in both loops; almost always exists since m ≪ N.

`apply_splice(…)`
Builds the merged sequence in O(N), only when accepted.

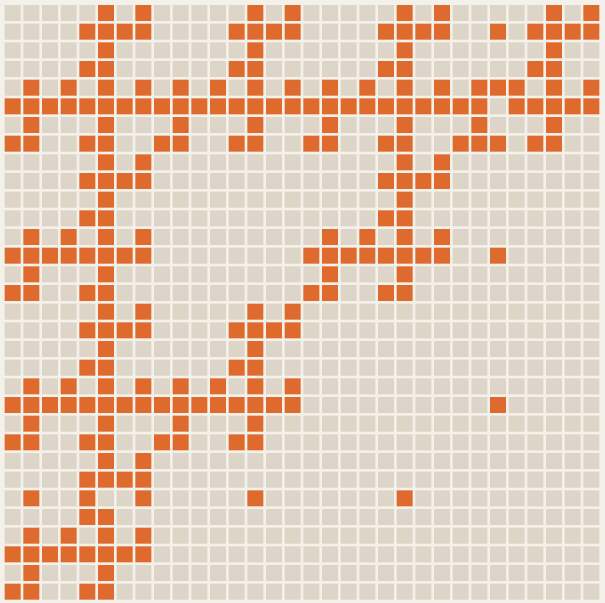
10.87% → 9.27%
At hc-p's optima: 112 improving inversions, 104 splices.

hc-p-rc LAYER S3 · LP REDUCED-COST GUIDANCE (PHASE 11)

S3: the LP says which pairs to use

ONCE **before the search starts**

One LP solve gives every pair its rc. **Orange**: 125 of 496 pairs, rc = 0. **Grey**: provably costs points.



EVERY ITERATION on the current circle; only step 2 differs from a plain HC

1 · Move type ▸ **2 · Cut points** ▸ **3 · Score Δ** ▸ **4 · Keep if Δ>0**

Step 2, guided (80% of inversions and splices)

a Sample 8 edges of the circle; take the worst by rc ▸ **b** Keep one endpoint; draw a partner profile with rc = 0 ▸ **c** Cut where that partner sits: it becomes the neighbour

Uniform step 2: two random cut positions (all swaps, 20% of the rest).

7.6% vs 0.56%
proposals that improve, guided vs uniform

74 → 10
points proven lost: score ≤ 748 − Σ|rc|

9.27% → 1.34%
gap to the optimum

ga-e LAYER S4 · EDGE-GENOME GA (PHASE 12)

S4: a genome of edges, not of order

Parent A · 14

	A	B	C	D
A	·	2	·	2
B		·	1	1
C			1	1
D				·

+

Parent B · 16

	A	B	C	D
A	·	1	2	1
B		·	1	2
C			·	1
D				·

→

Child · 18

	A	B	C	D
A	·	1	1	2
B		·	2	1
C			·	1
D				·

Cell = edges between two profiles (upper half shown); each profile has degree 2·n_p = 4.

- **1** keep edges both parents share
- **2** + parents' other edges, longest first
- **3** + greedy completion of degrees

4 connect components if needed

`edge_crossover(…)`
Steps 1–4 on the m × m tables; the child is always a valid circle.

`edge_mutation(…)`
One random 2-exchange that keeps the table connected.

`genome_fitness(…)`
Σ d·x: no circle needed until the very end (Euler).

15.06% → 10.61%
ga → ga-e; real parents 408 + 414 → child 502

ga-m OPTIONAL LAYER · MEMETIC (PHASE 12)

Memetic: local search on some children

Tournament
two parents

→

Edge crossover
S4, as before

→

Mutation
one 2-exchange

→

Local search
on 20% of children

→

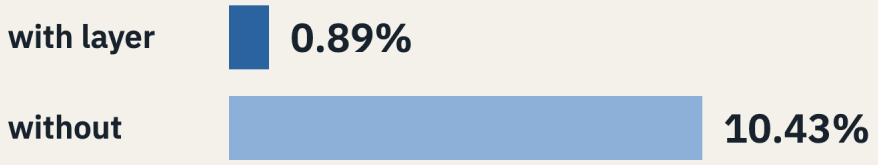
Next generation
duplicates rejected

`MemeticLayer(probability=0.2).improve(x, …)`

Runs the exact solver's bulk 2-exchange on the child's table: replace edge types (a, b), (c, d) with (a, c), (b, d) or (a, d), (b, c), moving **every copy** of a profitable pattern at once, best improvement first, until no move helps.

Lamarckian: the improved table replaces the child. A separate object, so `memetic=None` gives back the pure GA with the same seed.

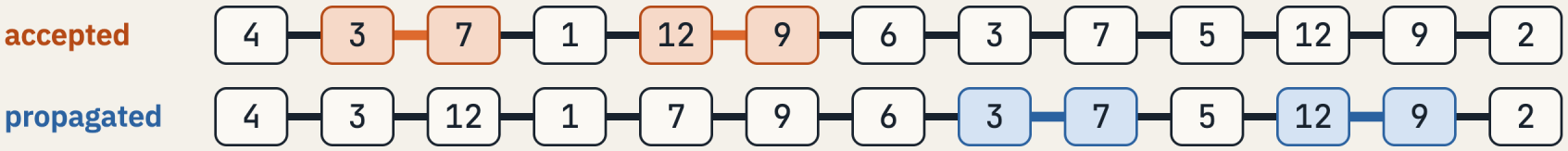
Gap at N = 160, same GA and seeds



At N = 1000 each child's local search is too slow for 0.2 s: strong at small size only.

`hc-p-prop` LAYER S5 · MOVE PROPAGATION (PHASE 13)

S5: repeat a winning move elsewhere



Cutting 3–7 and 12–9 and reversing between them adds 3–12 and 7–9. The gain needs only those four profiles: g = 4 + 3 − 1 − 2 = +4.

`Propagation.from_move(seq, i, j, crossed)`

Remembers the pattern: remove {p1, p2}, {q1, q2}; add p1–q1, p2–q2.

`.edges(seq, x, y) · .next_move(seq, rng)`

Finds the other edges of those types (a byte-substring search) and proposes the inversion or splice that repeats it: a normal check, up to 64 per trigger, stopping at the first failure.

A mixed result, kept on record

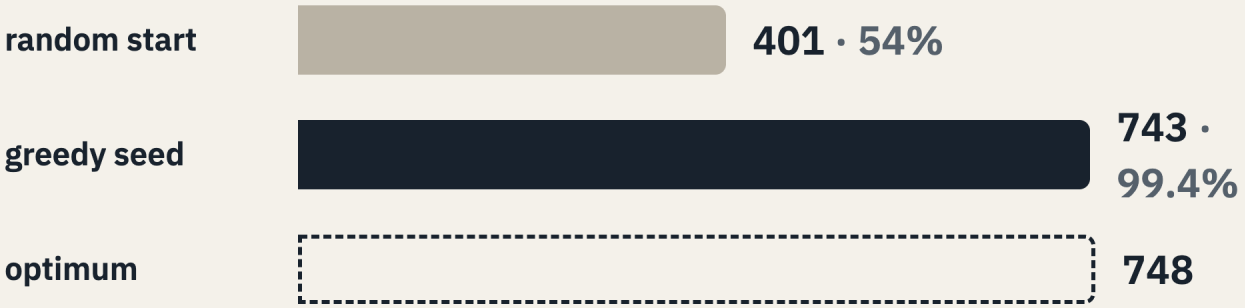
Instance	without	with S5
N = 160, C = 5	1.34%	1.43%
N = 1000, C = 5	4.6%	2.4%
N = 10,000	ref.	worse

At 10,000 each acceptance pays an O(N) scan, while most random moves still improve anyway.

`hc-p-grasp · ga-e-seed` LAYER S6 · GREEDY SEEDING (PHASE 14)

S6: start from greedy circles

Score of a starting circle, mean of 200, benchmark instance



1 Sort profile pairs by distance + noise → **2** Give each pair as many edges as degrees allow → **3** Merge components cheaply → **4** Euler circuit → circle

`greedy_multigraph → connect → euler_sequence`

`seed_sequence(state, rng)`

Steps 1–4, for every HC start and restart.

`seeding=0.1`

The GA builds 10% of its first population this way.

`double_bridge(seq, rng)`

0.00%

gap for both at N = 160; HC and GA still do the climbing and recombining.

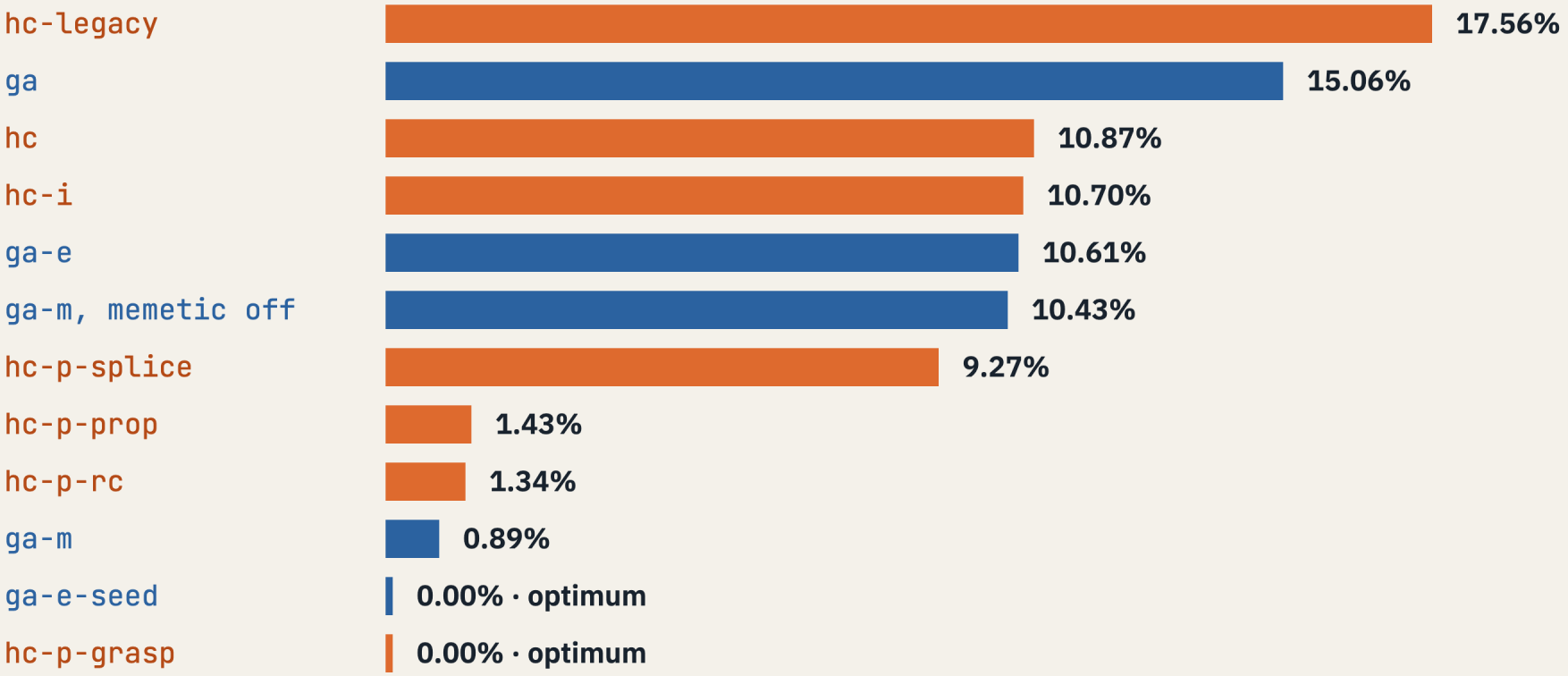
03

Compared results

Same instances, same seeds, 0.2 s per run for every algorithm; gap measured against the proven optimum.

N = 160 · C = 5 RESULTS · THE WALKTHROUGH INSTANCE

From 17.6% below optimum to 0%



748

proven optimum (78 ms)

Bars: mean gap over 3 seeds at 0.2 s each. **Orange** = HC family, **blue** = GA family.

BENCHMARK RESULTS · MEAN GAP TO OPTIMUM · 0.2 S · 3 × 3 RUNS

The gains hold as N and C grow

Version	What it adds	N=160, C=5	N=1000, C=5	N=1000, C=7
hc-legacy	the lab's HC	16.40%	35.12%	36.48%
ga	the lab's GA	14.72%	43.39%	43.65%
hc-i	productive restarts, 80% inversions	9.42%	17.20%	20.15%
hc-p-splice	profile state + splice move	9.25%	16.90%	19.60%
hc-p-rc	+ LP reduced-cost guidance	1.38%	4.55%	5.26%
hc-p-prop	+ move propagation	1.50%	2.36%	5.02%
ga-e	edge-genome crossover	9.85%	26.82%	17.81%
ga-m	+ memetic local search on 20% of children	0.60%	41.60%	35.60%
hc-p-grasp	+ greedy (GRASP) restarts	0.00%	0.00%	0.28%
ga-e-seed	edge crossover + greedy seeding	0.00%	0.03%	0.12%
exact	MILP on the profile table	0.00%	0.00%	0.07%

N = 10,000 · C = 8 RESULTS · BEYOND THE N × N MATRIX

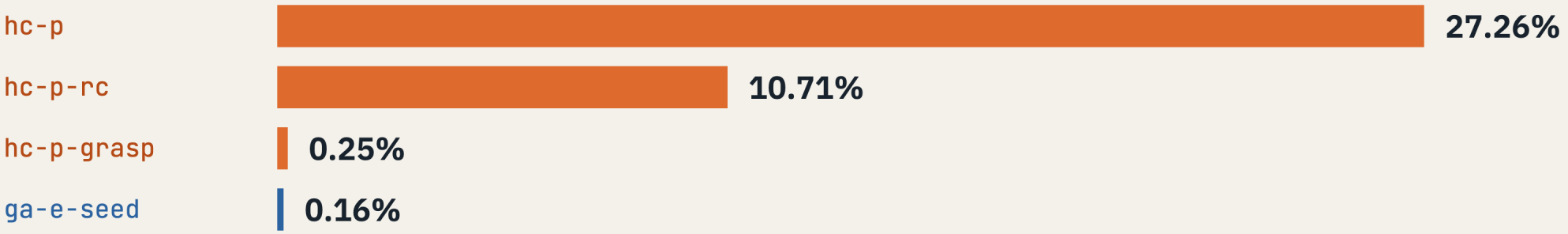
Where the classic versions cannot run

100,000,000
cells the classic N × N matrix would need

m = 256
distinct profiles: the table the layered versions use

0.70 s
to prove the optimum, 78,850, exactly

Gap to the optimum after 1 s per run



TAKEAWAYS

What I learned, and what comes next

1 Measure against the truth

Loose bounds hid a 7–15% gap; the exact solver turned “good enough” into a measurable problem.

2 Representation decides what a search sees

Profiles, not children, carry the information: splice, reduced costs, edge genomes, greedy seeds.

3 Use what the exact model knows

LP reduced costs are cheap and turn blind proposals into targeted ones: 9.3% → 1.4%.

4 Keep every version runnable

One-switch ablations showed what helped (S3, S6), what helped only sometimes (S5, memetic).

Next: larger C

All results use $5 \le C \le 8$, where most children share a profile. As C grows, profiles stop repeating ($m \approx N$): splice and aggregation lose their edge, while S3 and S6 should survive. Plan: map that regime, revise the exact model for $m \approx N$, re-test every layer.

Thank you. Questions?

Every version is runnable by name. Start from README.md, HISTORY.md and walkthrough.ipynb.

```
cd project
python -m circle_optimization list
python -m circle_optimization run hc-legacy --children 160 --optimum
python -m circle_optimization compare hc-legacy ga hc-i hc-p-grasp ga-e-seed
```